Learning-based Distance Computation Operators for High-Dimensional Vector Similarity Search

Zengyang Gong
Hong Kong University of Science and Technology
zgongae@cse.ust.hk

Yuxiang Zeng
State Key Laboratory of Complex & Critical Software Environment, Beihang University
yxzeng@buaa.edu.cn

Lei Chen
Hong Kong University of Science and Technology (Guangzhou)
Hong Kong University of Science and Technology
leichen@cse.ust.hk

ABSTRACT

Approximate k-nearest neighbor (AKNN) search for high-dimensional vectors is a critical task for enhancing the reliability and inference efficiency of Large Language Models (LLMs). However, the efficiency of nearest neighbor search is often hindered by the high computational cost of distance calculations in high-dimensional spaces. In this work, we harness the representational power of learning-based models to address this challenge. Specifically, we propose a Learning-based Distances CompUtation Operator (*L-DCUO*), which employs similarity-aware learning models and novel training strategies. Through this learning-based operator, high-dimensional vectors are projected into significantly lower-dimensional spaces, transforming the original distance computation problem into a low-dimensional operation. This substantially reduces computational overhead while preserving essential distance information. By integrating the proposed learning-based operator into the SOTA graph-based search index, we achieve a 3.7 × speedup in approximate high-dimensional vector similarity search. Overall, our method provides a promising alternative to existing approaches for efficient and scalable vector search.

PVLDB Reference Format:
Zengyang Gong, Yuxiang Zeng, and Lei Chen. Learning-based Distance Computation Operators for High-Dimensional Vector Similarity Search. PVLDB, 19(10): , 2026.

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/gzyhkust/SHG-Index.

1 INTRODUCTION

Vector databases [1, 2, 4] emerge as essential infrastructure supporting the rise of large language models (LLMs), owing to their ability to enable efficient data storage [26, 55], information retrieval [41, 42], and retrieval-augmented generation [11, 32]. A core operation within these systems is the search for k nearest neighbors in high-dimensional Euclidean spaces. To overcome the challenges posed by the "curse of dimensionality" [27, 43] in high-dimensional vector data and to strike a balance between accuracy and efficiency, recent research and real-world applications have mainly adopted approximate k-nearest neighbor (AkNN) search. By aiming for high recall while reducing search latency, AkNN methods provide a practical solution for real-world applications where efficiency and scalability are critical.

The *distance computation* dominates the computational cost of similarity search. The operator calculates and verifies whether the distance between vectors falls within a specific threshold, accounts for over 77% of the total processing time [18]. Existing works explore using learned models to navigate the searching process [31, 52]. A more prevalent and effective approach involves optimizing distance comparison operations [13, 18]. They commonly leverage transformation techniques (*e.g.*, orthogonal transformations) to sample a subset of dimensions for approximating exact distances. Although these methods successfully reduce computational overhead, they suffer from accuracy limitations, as the sampled dimensions may fail to capture the most informative features for similarity estimation. Furthermore, such random or fixed transformations lack adaptability to the underlying data distribution and task-specific requirements, limiting their effectiveness in diverse real-world scenarios.

Recent advances in learning-based models have demonstrated remarkable representational capabilities [7–9]. In this work, we leverage these capabilities to embed original high-dimensional vectors into a much lower-dimensional space, thereby reducing the number of dimensions scanned during distance computations and significantly decreasing AkNN search time cost. Specifically, we propose the Learning-based Distances CompUtation Operator (*L-DCUO*) to achieve this objective. To optimize operator performance, we carefully design the model architecture, implement a judicious training strategy, and further identify modules with suboptimal performance for targeted improvement through an active fine-tuning approach. To motivate our method, we present a motivating example and report preliminary experimental results below.

Motivation example. Fig. 1 shows the example. The original vectors x and y are both 6-dimensional. By applying a learned model, they are mapped into a 3-dimensional space, yielding vectors x' and y'. We compute the L_2 distance to measure the similarity. In the original space, the distance is calculated as $dis(x, y) = \sqrt{(|1-3|^2 + |2-4|^2 + \cdots + |6-8|^2)} = \sqrt{24}$. After embedding, the distance becomes $dis(x', y') = \sqrt{(|7-9|^2 + |2-4|^2 + |6-8|^2)} = \sqrt{24}$. Thus, the distance between vectors is preserved, while the

Proceedings of the VLDB Endowment, Vol. 19, No. 10 ISSN 2150-8097.
doi:

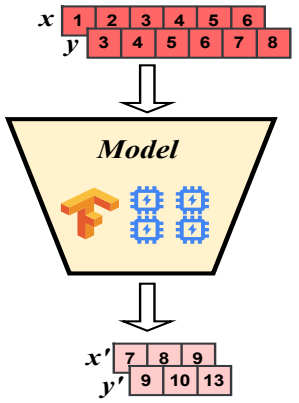
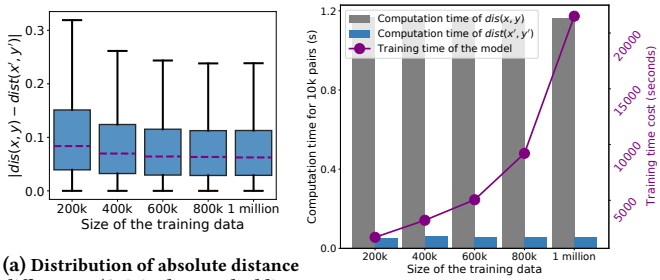


Figure 1: Motivation Example: The L_2-distance between two vectors remains unchanged after transformation *e.g.*, $dis(x, y) = dis(x', y') = \sqrt{24}$



(a) Distribution of absolute distance differences (Original vs. Embedding, *e.g.*,$|dis(x, y) - dis(x', y')|$)

(b) Computation time cost

Figure 2: Preliminary experimental results on *Gist* (960-dimensional vectors)

dimensionality is significantly reduced, resulting in lower computational cost for distance calculations. Our objective is to train a model that achieves this reduction while maintaining similarity. The following preliminary results demonstrate feasibility.

Preliminary Results: 32-Dimensional Embeddings Preserve Original 960-Dimensional Distance Information. Fig. 2 presents preliminary results on the widely adopted Gist dataset. We train a four-layer neural network to map the original 960-dimensional vectors into lower-dimensional representations (512, 256, 128, and finally 32 dimensions). Fig. 2a shows the distribution of absolute distance differences (*e.g.*, $|dis(x, y) - dis(x', y')|$) for 10,000 randomly sampled vector pairs, evaluated across different training data sizes. Fig. 2b reports the distance computation time for these pairs and the model training time as the training data size increases. **The results show that the 32-dimensional embeddings achieve over 20 times faster distance computation, while maintaining an average distance error below 0.1.**

However, as indicated by the preliminary results, 3 main challenges need to be addressed: (1) *Designing an effective embedding model architecture* (Sec. 3.1): In the worst cases, the distance error can exceed 0.2, even with a training dataset of one million samples. (2) *Developing more efficient training strategies* (Sec. 3.2): As the training data size increases, the training time rises significantly, reaching nearly 7 hours. (3) *Implementing active fine-tuning methods* (Sec. 4): Simply increasing the training data size leads to only marginal improvements, with model performance plateauing as the dataset grows from 600,000 to one million samples.

Contribution. The main contributions are listed as follows:

- We are the first to propose and design an effective learning-based model specifically as a Distances CompUtation Operator (*DCUO*) for high-dimensional vector similarity search.
- We develop a similarity-decoupled embedding architecture that enhances the representational capacity of the learning model. Furthermore, we design an efficient training strategy that accelerates model convergence by 2.7 ×.
- We introduce an active fine-tuning strategy that targets underperforming modules within the learning-based model for optimization. Compared with the model without fine-tuning, this strategy yields a performance improvement of up to 24.66%.
- We conduct extensive experiments on real-world datasets across multiple similarity metrics. When integrated into SOTA graph-based indices, our method achieves 3.7× speedup in search time. Moreover, compared with the existing Distance Comparison Operator (DCO), our approach delivers a 3.1 × speedup in distance computations.

Roadmap. The rest of this paper is organized as follows. Sec. 2 describes how the basic learning-based model functions as a distance computation operator. In Sec. 3, we present our model architecture and the associated training strategy. The active fine-tuning approach is detailed in Sec. 4. Finally, experiments, related work, and conclusions are presented in Sec. 6–Sec. 8.

2 PRELIMINARIES

This section formalizes the Learning-based Distance CompUtation Operation (*L-DCUO*) problem. We first introduce the standard Distance CompUtation Operation (DCUO) over high-dimensional vectors in Sec. 2.1 and then defines the L-DCUO problem in Sec. 2.2.

2.1 Standard Distance Computation Operation Over Vector Data: Definition and Limitation

The standard distance computation operation, which calculates similarity between high-dimensional vectors, is a fundamental operation in vector similarity search and serves as the primary efficiency bottleneck in vector retrieval systems.

2.1.1 Formal Definition. High-dimensional vectors are generated by applying embedding models to unstructured data (*e.g.*, images and text). This data type enables semantic search and forms the backbone of advanced AI systems like large language models (LLMs). Below is the formal definition of vector data.

Definition 1 (Vector). *A vector o is defined as a high-dimensional point in the d-dimensional space $\mathbb{R}^d$, denoted as $o = (x_1, \cdots, x_d)$. For convenience, we denote the i-th coordinate of o by $o[i]$, i.e., $o[i] = x_i$.*

A vector dataset $\mathcal{D}$ is denoted as a collection of n vectors, *i.e.*, $\mathcal{D} = \{o_1, \cdots, o_n\}$. The similarity between any two vectors is quantified by a distance function $dis(\cdot, \cdot)$. Common similarity functions

include L_2 distance and inner product. For ease of presentation, we adopt L_2 distance as the default similarity metric throughout this work, and discuss extensions to other similarity functions in Sec. 6.

Definition 2 (Standard Distance Computation Operation)**.** *Given a (query) vector $q \in \mathbb{R}^d$ and a (candidate) vector $o \in D$, the standard distance computation operation (DCUO) scans all d dimensions to compute the similarity using the L_2 distance (by default):*

$$dis\,(o, q) = \sqrt{\sum_{i=1}^{d} \Big(q[i] - o[i]\Big)^2} \tag{1}$$

This operation serves as the core computation primitive for Approximate k-Nearest Neighbor (AkNN) search, which is formally defined as follows:

Definition 3 (Approximate k-Nearest Neighbor Search)**.** *Given a query vector $q \in \mathbb{R}^d$ and a vector dataset $\mathcal{D} = \{o_1, \ldots, o_n\} \subseteq \mathbb{R}^d$, vector similarity search aims to retrieve the top-k nearest vectors $\mathcal{S} \subseteq \mathcal{D}$ to q such that any $o^* \in \mathcal{S}$ satisfies $dis(q, o^*) \leq dis(q, o)$ for all $o \in \mathcal{D} \setminus \mathcal{S}$.*

2.1.2 Limitations of Prior Solutions. Since distance computations dominate the runtime of vector similarity search, prior work on DCUO focused on reducing the number of scanned dimensions while preserving distance information. Despite these efforts, existing methods suffer from the following dilemma in *balancing retrieval accuracy and computational efficiency*:

- **Exact algorithms** rely on a *scan-from-scratch* strategy that calculates distances by scanning all dimensions. Although this ensures full accuracy, it results in poor scalability, as computational cost grows linearly with dimensionality.
- **Approximate Algorithms** utilize techniques such as quantization or PCA to accelerate computation by estimating distances. While this improves efficiency, the introduced approximation often leads to non-negligible degradation in the recall rates of the final results.

2.2 Learning-Based Distance Computation Operation: Definition and Naive Solution

To address this limitation, we are motivated to formulate a new problem: Learning-based Distance CompUtation Operation (*L-DCUO*).

2.2.1 Formal Definition. We aim to design a learning-based approach that embeds high-dimensional vectors from $\mathbb{R}^d$ into a much lower-dimensional space $\mathbb{R}^{d'}$, where $d' \ll d$. By performing distance computations in $\mathbb{R}^{d'}$ rather than $\mathbb{R}^d$, this DCUO method can significantly improve the computational efficiency. Crucially, this method must preserve the relative distance relationships among vectors in the original space to ensure retrieval accuracy. Therefore, our objective is to develop *learned metric embeddings* that maintain these ordinal relationships and enabling both efficient and accurate similarity computation in the compressed representation space.

Accordingly, we formally define the L-DCUO problem as follows.

Definition 4 (Learning-based Distance CompUtation Operation (*L-DCUO*))**.** *Given a vector dataset $\mathcal{D} \subseteq \mathbb{R}^d$, the L-DCUO problem asks to learn an embedding $M : \mathbb{R}^d \to \mathbb{R}^{d'}$ with $d' \ll d$, such that for any query vector $q \in \mathbb{R}^d$ and data vector $o \in \mathcal{D}$, the Euclidean distance $\widetilde{dis}(q, o)$ between their embeddings $M(q)$ and $M(o)$ approximates their true distance:*

$$\widetilde{dis}(q, o) = dis\Big(M(q), M(o)\Big) = \sqrt{\sum_{i=1}^{d'} \Big(q'[i] - o'[i]\Big)^2}, \tag{2}$$

The learned model M should minimize the discrepancy between the true distances in the original space and the approximated distances in the embedded space:

$$\min_{M} \sum_{o \in \mathcal{D}} \Big| dis(q, o) - dis\Big(M(q), M(o)\Big) \Big| \tag{3}$$

2.2.2 A Naive Solution. A straightforward approach to train the embedding model M is to directly minimize the Mean Squared Error (MSE) between pairwise distances in the original space and those in the embedded spaces:

$$\mathcal{L}(x, y) = |dis(M(x), M(y)) - dis(x, y)|^2 \tag{4}$$

In practice, we randomly sample pairs of vectors x and y from the vector dataset (D) and compute their exact distance $dis(x, y)$. Each training sample is formed as a triplet $(x, y, dis(x, y))$. The model parameters are then updated via standard gradient descent with learning rate α to minimize the empirical loss over these samples.

Limitation. While this straightforward strategy is simple to implement, our preliminary experiments reveal a critical limitation: *the model's accuracy plateaus after a moderate amount of training data, and further sample expansion fails to improve search performance while substantially increasing training time.*

As illustrated in Fig. 2, the training loss converges and stabilizes with increased data volume, showing no further improvement. However, this marginal benefit comes at a substantial cost: training time grows dramatically with more samples. To address this limitation, the next section introduces a novel model architecture and an improved training strategy to accelerate convergence.

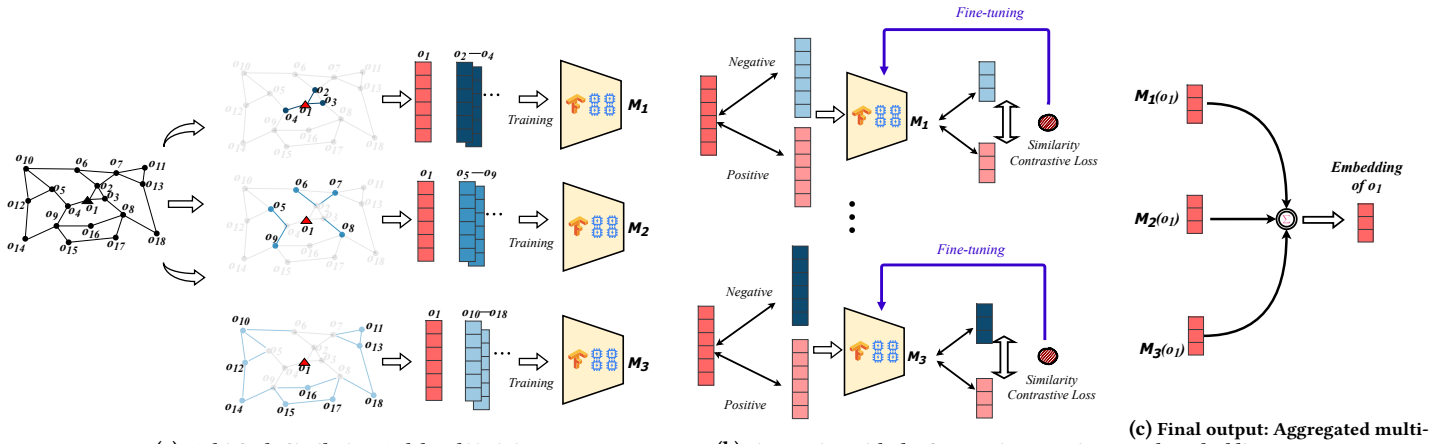
3 GRAPH-GUIDED EMBEDDING FRAMEWORK

This section first presents the similarity-decoupled embedding architecture (Sec. 3.1), which maps high-dimensional vectors to a lower-dimensional space while preserving distances. We then introduce our multi-scale training strategy (Sec. 3.2) dedicated to this embedding architecture.

3.1 Similarity-Decoupled Embedding Architecture

3.1.1 Preliminary: Graph Index for ANN Search. Graph-based indexing is a leading approach for ANN search in vector databases, offering a good balance between search efficiency and recall [5, 6, 38, 46]. Among graph-based indexes, Navigable Small-World (NSW) and its variants [17, 23, 33, 35, 57] have become standard in vector databases and search engines [11, 38, 47]. Their small-world property ensures distant nodes are connected within few hops.

Formally, a vector dataset V is indexed as a proximity graph $G = (V, E)$, where each node represents a vector and edges E connect vectors based on their proximity. Low-hop connections preserve local search accuracy, while long-range edges bridge distant regions to enable rapid navigation. For a query q, a greedy



(a) Multi-Scale Similarity Model and Training

(b) Fine-tuning with the Contrastive Learning

(c) Final output: Aggregated multi-scale embeddings

Figure 3: Overview of the Learning-based Distance Computation Operator

traversal algorithm progressively moves to the neighbor locally closest to q until convergence. This approach achieves high recall in typically $O(\log n)$ time, where n is the number of vectors.

Example 1. Fig. 3a is a NSW graph of vectors o_1–o_{18}. To search the nearest neighbor (*e.g.*, of o_1), a greedy traversal procedure starts from an entry node and iteratively moves to the neighbor closest to the query vector until no closer neighbor can be found.

3.1.2 Multi-Scale Embedding Model. As mentioned in Sec. 2.2.2, a naive embedding solution typically learns a monolithic similarity measure across all distance scales. This leads to *diminishing returns in training*: as the training dataset grows, training time increases significantly while accuracy improvements become marginal, ultimately limiting search performance. Two key factors contribute to this **limitation**: (1) in large-scale vector databases, pairwise distances span multiple orders of magnitude, which a single model cannot faithfully represent; and (2) learning solely from vector-distance pairs ignores the topological structure of the underlying graph index, causing a mismatch with the actual retrieval process.

Main Idea. To address these limitations, we propose a **Similarity-Decoupled Embedding Architecture**. The main idea is to decompose embedding learning into distinct topological scales defined by graph hop distance. The overall model M comprises multiple independent sub-modules $\{M_1, M_2, \ldots, M_L\}$, where L is a hyperparameter. Each sub-module M_l specializes in capturing similarity patterns at a specific hop range.

To train M_l, we partition the dataset based on graph topology. Specifically, for a vector o, the training set for M_l is defined as:

$$\mathcal{H}_l(o) = \left\{ v \in V \setminus \{o\} \;\middle|\; l \cdot \left\lfloor \frac{maxHop}{L} \right\rfloor \leq hop(o,v) < (l+1)\left\lfloor \frac{maxHop}{L} \right\rfloor \right\} \tag{5}$$

where $hop(o, v)$ is the shortest hop distance from o to v in graph G, and $maxHop$ is the farthest hop distance from o to any node in G.

Each sub-module M_l is trained only on pairs (o, v) with $v \in \mathcal{H}_l(o)$, allowing it to specialize in a particular hop range. The final embedding is obtained by a weighted aggregation of these multi-scale representations from all L sub-modules.

Example 2. Fig. 3b and Fig. 3c illustrate an example of the embedding model M decomposed into three sub-modules M_1–M_3. For vector o_1, the dataset is partitioned into three sets: $\mathcal{H}_1(o_1) = \{o_2, o_3, o_4\}$, $\mathcal{H}_2(o_1) = \{o_5, o_6, \ldots, o_9\}$, and $\mathcal{H}_3(o_1) = \{o_{10}, o_{11}, \ldots, o_{18}\}$, corresponding to its 1-hop, 2-hop, and 3-hop neighbors, respectively. Each sub-module M_l are trained using vector-distance pairs from $\mathcal{H}_l(o_1)$, and their outputs are aggregated via a weighted sum to produce the final embedding of o_1.

3.2 Multi-Scale Similarity Training Strategy

Landmark-based Training Data Selection. Constructing an effective training dataset poses an essential challenge, as the total number of possible vector pairs grows quadratically, $O(N^2)$, for a collection of N vectors. This makes the exhaustive utilization of all vector pairs computationally intractable. Furthermore, our preliminary experiment shows that simply increasing training data volume yields diminishing returns: although computational overhead rises substantially with data size, the marginal improvement in predictive accuracy quickly plateaus. For example, adding 200,000 training samples increased training time by 3.41 hours while reducing average computation error by less than 0.001. This observation underscores the need for a strategic selection mechanism that identifies a concise yet informative subset of training pairs to balance efficiency with model performance.

To this end, we propose a **landmark-based training sample selection** strategy. Within the underlying graph index structure, certain nodes are significantly more representative of the global topology than others. These nodes are referred to as *landmarks* [20, 21, 29]. Let U denote a set of selected landmarks, where $|U| \ll N$. By restricting training to pairs involving these landmarks, the total number of training samples is reduced to $O(|U|^2)$. We employ the *farthest point sampling* strategy [20] to select representative landmarks from the graph. This approach iteratively selects the vertex farthest from the current set of landmarks, aiming to identify regions of the graph that are not yet well-covered.

Progressive Training Strategy. To accelerate convergence and ensure stable training across different topological scales, we propose

a progressive training strategy tailored to the multi-scale architecture. This approach partitions the training process into distinct phases, each targeting a specific similarity range and its corresponding sub-module. For each selected landmark u, we utilize the multi-scale neighbor sets $\{\mathcal{H}_L(u), \mathcal{H}_{L-1}(u), \cdots, \mathcal{H}_1(u)\}$ constructed during training data selection. Training proceeds from the coarsest scale L down to the finest 1. In each phase l, the module M_l is trained to preserve distance information between u and the vectors in $\mathcal{H}_l(u)$. The optimization is guided by the following loss function,

$$\mathcal{L} = \left| \frac{1}{L} \sum_{l=1}^{L} \left(dis(M_l(u), M_l(o_j)) - dis(u, o_j) \right) \right|^2 \tag{6}$$

where u is a selected landmark and o_j is an associated neighbor vector. The loss minimizes the discrepancy between distances in the original and embedding spaces.

Furthermore, to facilitate knowledge transfer across different scales, the parameters of the subsequent module M_{l-1} are initialized with the weights of the trained M_l. It is then fine-tuned on the finer-grained neighbor set $\mathcal{H}_{l-1}(u)$. This warm-start mechanism facilitates the transfer of coarse-scale representations to finer-scale specialization. This process repeats sequentially until the module M_1 is fully trained.

Algorithm Details. Algo. 1 details our training algorithm. Specifically, Lines 1–3 describe the landmark-based training data selection phase: *Landmarks* are first selected from the graph index, and multi-scale neighbors are then built for each landmark. The progressive training phase is presented in Lines 4–11. Iterating from $l = L$ down to 1, the algorithm first initializes the modules $\{M_k\}_{k=1}^{l}$ using the weights from the previously trained M_{l+1} (Lines 5–6), ensuring a warm start for subsequent training. For each neighbor o_j in $\mathcal{H}_l(u)$, embeddings are generated using M_l, and both the original distance $dis(u, o_j)$ and embedding-space distance $dis(M_l(u), M_l(o_j))$ are computed (Lines 7–9). The loss in Eq. (6) is then calculated in Line 10, and the parameters of M_l are updated via gradient descent with a similarity-adaptive learning rate α_l, which assigns larger values to more similar vector pairs to prioritize fine-grained distinctions during training. Finally, the fully trained embedding model M, comprising all sub-modules, is returned.

Example 3. As illustrated in Fig. 3(b)-(c), let o_1 be the selected landmark. The training process is divided into three phases. In Phase 1, the training begins with module M_3 using the coarsest neighbor set $\mathcal{H}_3(o_1) = \{o_{10}, o_{11}, \ldots, o_{18}\}$ and a learning rate $\alpha_3 = \frac{1}{3}\alpha$. Subsequently, the weights of M_2, M_1 are initialized using the trained M_3. In Phase 2, M_2 is further trained using $\mathcal{H}_2(o_1) = \{o_5, o_6, \ldots, o_9\}$ with $\alpha_2 = \frac{1}{2}\alpha$. Similarly, M_1 inherits weights from M_2. In the final Phase, M_1 is trained with the nearest neighbors $\mathcal{H}_1(o_1) = \{o_2, o_3, o_4\}$ using learning rate $\alpha_1 = \alpha$. During this process, the loss is calculated as $Loss = \left| \frac{1}{3} \sum_{l=1}^{3} \left(dis(M_l(o_1), M_l(o_j)) - dis(o_1, o_j) \right) \right|^2$, where o_j represents a vector chosen from the training data.

4 ACTIVE FINE-TUNING STRATEGY

While the progressive training strategy effectively initializes and trains all modules in the embedding model M, certain modules may still underperform in capturing cross-scale relationships. To address this limitation, this section presents our active fine-tuning strategy. We first motivate the need for targeted refinement, then introduce a method to identify underperforming modules, and finally present a contrastive learning based approach to fine-tune these modules.

Algorithm 1: Multi-Scale Similarity Training Method

Input: Graph index G built from vector dataset $\mathcal{D}$, the number L of sub-modules, and learning rate α
Output: Trained embedding model M

```
// Landmark-based Training Data Selection
```
1 $U \leftarrow$ select landmarks from the graph index G;
2 **foreach** *landmark* $u \in U$ **do**
3 Build multi-scale neighbors $\mathcal{H}_1(u), \mathcal{H}_2(u), \cdots, \mathcal{H}_L(u)$ by Eq. (5) ;
4 **for** $l \leftarrow L$ **to** 1 **do** `// Progressive Training`
5 **if** $l < L$ **then**
6 Initialize $\{M_k\}_{k=1}^{l}$ with weights of M_{l+1};
7 **foreach** *vector* $o_j \in \mathcal{H}_l(u)$ **do**
8 $M_l(u), M_l(o_j) \leftarrow$ generate embeddings of vectors u, o_j with module M_l;
9 $dis(u, o_j), dis(M_l(u), M_l(o_j)) \leftarrow$ calculate distances in original and embedding spaces;
10 $loss \leftarrow$ calculate the loss by Eq. (6), $\alpha_l \leftarrow \frac{\alpha}{l}$;
11 $M_l \leftarrow M_l - \alpha_l * \frac{\partial Loss}{\partial M_l}$;
12 **return** *Embedding model* M;

4.1 Motivation and Key Idea

Motivation. The progressive training strategy enables each module M_l to specialize in preserving distances at a specific similarity scale. While this specialization is necessary, it introduces a critical **limitation**: a module may become overly tuned to its training distribution at the expense of cross-scale generalization. In other words, a module that performs well on its target neighbors may exhibit notably higher error when encoding relationships between vectors from different scales.

This tension between *specialization* and *generalization* lies at the heart of embedding learning for large-scale high-dimensional vector data. An ideal embedding model should not only preserve local neighborhood structures, but also maintain coherent global geometry. When a module underperforms on cross-scale validation, it indicates a failure to balance these two objectives, a bottleneck that limits overall model quality.

Key Idea. This observation inspires our active fine-tuning method. Rather than blindly fine-tuning all modules, we first identify underperforming ones through cross-scale validation. We then refine these modules using contrastive learning, an intuitive design choice: to pull representations of similar vectors closer together while pushing dissimilar ones apart. The graph index provides ideal supervisory signals for this goal: ANN queries supply positive samples, while cross-scale validation sets serve as negatives.

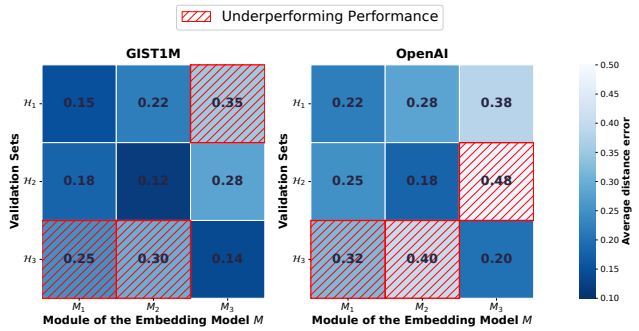


Figure 4: Identifying underperforming modules via cross-scale validation on GIST1M and OpenAI datasets

4.2 Identifying Underperforming Modules

A module that perfectly preserves distances for its target neighbors but fails on vectors from other scales has learned a local metric that does not align with the global data manifold. Such modules are "**underperforming**" not because they are poorly trained, but because they lack the **robustness** needed for cross-scale generalization. Identifying these modules requires evaluating them outside their comfort zone on vectors whose similarity relationships were not emphasized during training.

To quantify this, for each module M_l, we compute its distance preservation error on a validation set composed of vectors from *other* scales: $\{\mathcal{H}_i(u) \mid 1 \le i \le L, i \ne l\}$. We calculate the discrepancy between the exact distances and the estimated embedding distances for these validation samples. A module exhibits underperformance if it has a high discrepancy error. Modules meeting this criterion are flagged for subsequent fine-tuning, as they represent bottlenecks in the model's overall generalization ability.

4.3 Contrastive Fine-tuning via ANN Feedback

Contrastive learning turns a core intuition into practice: *similar vectors should be close in the embedding space, while dissimilar vectors should be far apart.* We apply this principle using the graph index G to supply training signals for fine-tuning underperforming modules.

Constructing Positives and Negatives. The graph index G provides rich supervisory signals for this contrastive objective. For each landmark u, we perform an Approximate Nearest Neighbor (ANN) search on G. The neighbors retrieved, denoted $ANN(G, u)$, serve as positive examples. In contrast, vectors located outside this landmark's neighbor set $\mathcal{H}_l(u)$ are treated as negative examples, denoted as $\mathcal{N}_l(u) = \bigcup_{i \ne l} \mathcal{H}_i(u)$.

Similarity-aware Contrastive Loss. We formulate a contrastive loss that encourages the embedding to respect the graph-indexed relationships. For a positive sample $u_i^+ \in ANN(G, u)$, the loss is defined as:

$$\begin{aligned} \mathcal{L}_i^{scl} &= -\log \frac{exp(\tau / dis(M_l(u_i^+), M_l(u)))}{\mathcal{P} + \mathcal{N}}, \\ \text{where } \mathcal{P} &= \sum_{u_i^+ \in ANN(G,u)} exp(\tau / dis(M_l(u_i^+), M_l(u))), \\ \mathcal{N} &= \sum_{u_i^- \in \mathcal{N}_l(u)} exp(\tau / dis(M_l(u_i^-), M_l(u))) \end{aligned} \tag{7}$$

where $dis(M_l(u_i^+), M_l(u))$ denotes the distance between the landmark u and a positive sample u_i^+, and $dis(M_l(u_i^-), M_l(u))$ denotes the distance between the landmark u and a negative sample u^-. The temperature parameter τ controls the sharpness of the contrastive distribution. For a mini-batch of size N_b, the overall loss is computed as $\mathcal{L} = \frac{1}{N_b} \sum_{i=1}^{N_b} \mathcal{L}_i^{scl}$.

5 *L-DCUO*: A PLUG-IN ACCELERATOR FOR VECTOR SIMILARITY SEARCH

Graph-based indices become the dominant paradigm for vector similarity search due to their superior trade-offs between result recall and search latency. However, traversing graph indices inherently involves a massive number of distance computations between the query vector and the visited nodes. In this work, we accelerate this process by replacing the exhaustive distance operation with our *L-DCUO* method. *L-DCUO* significantly reduces the dimensionality scanned per calculation by mapping high-dimensional vectors into lower-dimensional embeddings using the trained model.

Plug-In Integration. *L-DCUO* functions as a drop-in replacement for exact distance computations in graph-indexed vector similarity search. Taking the Navigable Small World (NSW) graph [34, 35] as an example. *L-DCUO* is used to compute the distance between the query vector and each newly visited node. this distance then serves as the criterion for updating the candidate result set. As an efficient, self-contained operation, *L-DCUO* can be seamlessly integrated into the SOTA graph index without modifying their core traversal routines. We empirically demonstrate this versatility in Sec. 6.

Algorithm Details. Algo. 2 details of the optimized graph search using *L-DCUO*. In the pre-processing phase, Lines 1-4 train the embedding model M using the progressive training strategy (Algo. 1), followed by fine-tuning underperforming sub-modules in M.

Lines 5-13 present the optimized graph traversal procedure using *L-DCUO*. The search begins at the entry node ep and maintains a k-sized max-heap W to track the current k nearest candidates, along with the distance threshold dis_k for the current k-th NN. For each visited node o, we examine every neighbor v of o and compute the approximate distance $dis(M(v), M(q))$ using *L-DCUO* (line 111), where $M(v)$ and $M(q)$ denote the coordinates of vectors v and q in the embedding space. If this distance falls below the current threshold dis_k, v is inserted into W and dis_k is updated accordingly. This process continues until the heap W is exhausted, resulting in the final k nearest neighbors.

6 EXPERIMENTAL STUDY

We first outline the experimental setup in Sec. 6.1, then evaluate the search performance of our method under varying k values and

Algorithm 2: Learning-based Vector Similarity Search

Input: Graph index G built from vector dataset $\mathcal{D}$, a query vector q, and a query parameter k
Output: Approximate k-nearest neighbors A to q

```
// Train and fine-tune the embedding model
```
$M \leftarrow$ Initialize and train the embedding modules $\{M_1, M_2, \cdots M_L\}$ by Algo. 1 ;
foreach $M_i \in M$ **do**
  if *performance of* M_i *is suboptimal* **then**
    Fine-tune M_i utilizing loss Eq. (7);

```
// Search with Learning-based Distance Computation Operator
```
$ep \leftarrow$ the entry node of G;
Start searching kNN to q from ep using a k-sized heap W;
$dis_k \leftarrow$ track kth nearest distance from vectors in W to q;
while $|W| > 0$ **do**
  $o \leftarrow$ select the nearest vector in W to q;
  foreach *vector* $v \in o$*'s neighbourhood* **do**
    $dis \leftarrow dis(M(v), M(q))$ using *L-DCUO*;
    if $dis \leq dis_k$ **then**
      Push vector u into W and update dis_k;
return $A \leftarrow$ *kNN to q among all vectors popped from* W;

Table 1: Summary of datasets

Dataset	Dim.	Card.	#(Query)
OpenAI	1,536	1M	10,000
GIST1M	960	1M	1,000
SIFT100M	128	100M	10,000
Deep100M	96	100M	10,000

distance metrics in Sec. 6.2, and finally conduct the ablation study in Sec. 6.3.

6.1 Experimental Setup

Datasets. We evaluate our proposed method using four real-world datasets characterized by various cardinalities and dimensions, as detailed in Table 1. These datasets are standard benchmarks in vector search research [5, 6, 38, 40, 46, 47]. Each dataset includes a test set of query vectors.

Baselines. We compare our proposed methods with four categories of existing methods for ANN search: ① Flat graph-based indexes: HNSW [35]; ② Vector dimensionality reduction methods: SEANet* [48], PM-LSH [58]; ③ Vector distance operations: ADSampling [18], DADE [13]; ④ Learning-based vector search methods: ROTLE [51]. The detailed implementations are outlined below, and all parameter settings follow the original papers.

- **HNSW** [35]: This is a pioneering graph-based index designed for ANN search. We set $M = 48$, $efConstruction = 80$.
- **SEANet*** [48]: This method proposes a novel autoencoder, based on the principle of Sum of Squares, to solve embedding approximation for data series. The model is trained using SGD, and α is selected from $\{0.1, 0.25, 0.5, 1, 1.25\}$.
- **PM-LSH** [58]: This method reduces dimensionality via random projections, based on the Johnson-Lindenstrauss lemma. We set the parameters to $m = 4$ and $\delta = \alpha = 1/e$.
- **ADSampling** [18]: This method proposes a vector distance comparison operation that uses random projection to determine if the exact distance exceeds a given threshold. We set $\epsilon = 2.1$ and $\Delta_d = 32$.
- **DADE** [13]: This method utilizes orthogonal transformation to improve ADSampling, where the variance of each dimension in the projected space is ordered from largest to smallest. We set $P_s = 0.1$ and $\Delta_d = 32$.
- **ROTLE** [51]: This method partitions space into a K-ary tree, where query embeddings match shared node embeddings for retrieval. We set $\alpha = 1.5$ and utilize a 256-ary tree structure.
- ***L-DCUO*** (proposed method) : Following previous studies [13, 18], our method is deployed on the HNSW index. The proposed training and fine-tuning are performed on the base-level graph, which contains all vectors. For the search procedure, pairwise distances are approximated using the low-dimensional representations produced by the *L-DCUO* model. We set $L = 3$.

Evaluation Metrics and Implementation. To evaluate search accuracy, we employ *time-recall curve*, reporting both average search time and Recall@k averaged across all queries. Experiments are conducted on a server equipped with four NVIDIA H800 GPUs and 40 Intel Xeon E5 (2.30 GHz) CPU cores.

6.2 Search Performance

Search time vs. Recall.. Fig. 5 shows the Recall-QPS trade-off performances of *L-DCUO* compared against baselines across four real-world datasets with $k = 20$. Our method consistently outperforms all competing approaches, achieving 1.6-3.7 × higher speeds than flat graph indices (*e.g.*, HNSW) and 1.2-2.1 × higher speeds than existing distance comparison optimizations (*e.g.*, DADE, ADSampling) at high recall targets ($Recall > 0.9$). *L-DCUO* attains significantly higher query throughput (QPS) while maintaining the same level of search accuracy, demonstrating the superior efficiency of our proposed indexing and search strategy.

Specifically, the performance gap is most pronounced on the GIST1M and SIFT100M datasets, where our method surpasses the existing distance comparison operations (*e.g.*, DADE, ADSampling) by a substantial margin. For instance, on GIST1M, *L-DCUO* more than doubles the QPS of the nearest competitor at high recall levels ($Recall > 0.8$). Even on the challenging Deep100M and OpenAI datasets, *L-DCUO* maintains a clear lead over strong baselines like ROTLE. These results validate that *L-DCUO* effectively reduces the computational cost of vector search without compromising retrieval quality, making it highly suitable for large-scale, latency-sensitive applications.

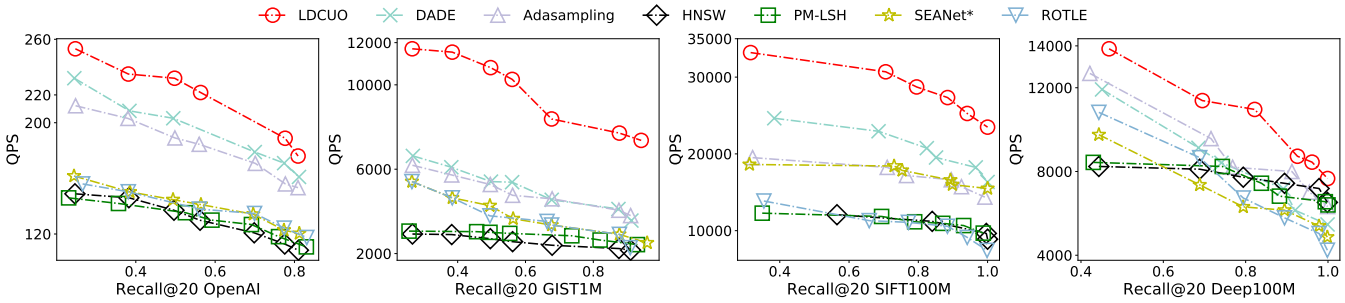


Figure 5: **Search performance on four real-world datasets ($k = 20$)**

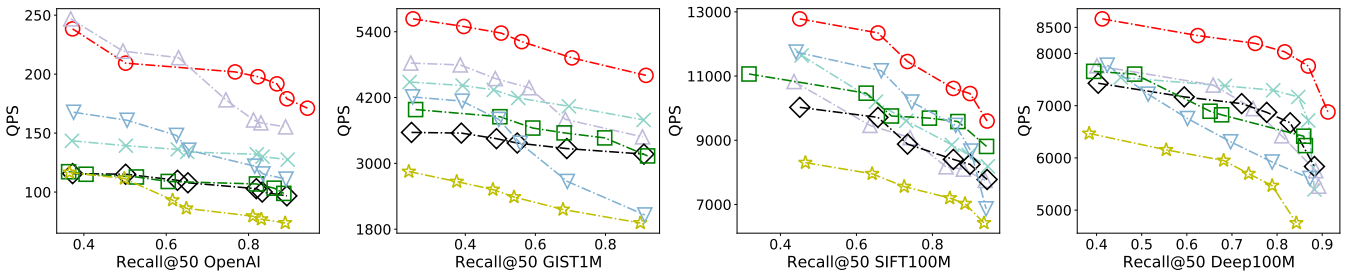


Figure 6: **Search performance on four real-world datasets ($k = 50$)**

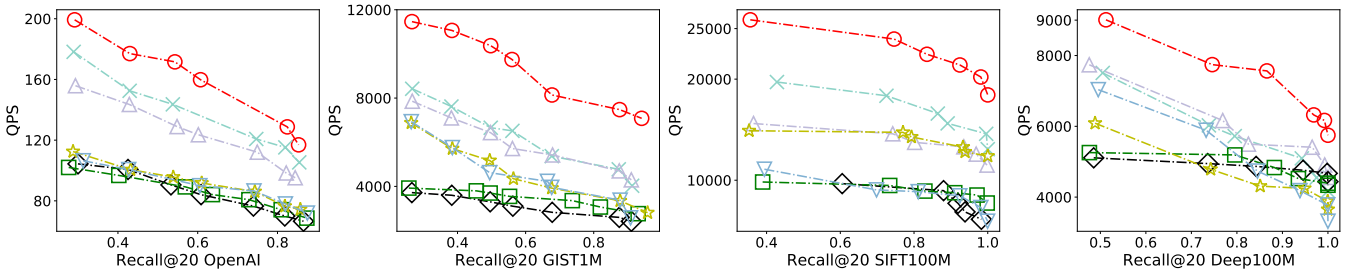
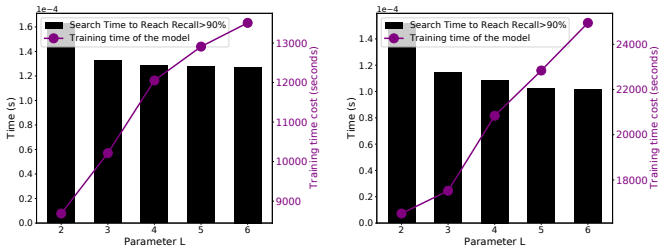


Figure 7: **Search performance on Inner Product Metric**

Effect of parameter k. Fig. 6 shows the search performance when k is increased to 50. Our proposed method maintains superior efficiency compared to existing state-of-the-art techniques. Specifically, at high recall targets ($Recall > 0.9$), *L-DCUO* is 1.5 - 2.9× faster than flat graph indices (*e.g.*, HNSW) and 1.6 - 1.9× faster than existing distance comparison operations (*e.g.*, DADE, ADSampling). While the throughput of competing methods drops noticeably as the retrieval burden increases, *L-DCUO* exhibits superior robustness, consistently maintaining the highest QPS across all four datasets. This stability indicates that our distance comparison optimization effectively mitigates the computational overhead even when a larger number of candidates need to be retrieved.

In terms of dataset-specific performance, the dominance of *L-DCUO* remains substantial on the GIST1M and Deep100M datasets. As observed in Fig. 6, on GIST1M, the performance maintains a significant gap above the baselines, achieving 1.5 × more throughput of the nearest competitor at various recall levels. Furthermore, on the challenging OpenAI and Deep100M datasets, where baseline methods struggle to maintain high QPS at high recall, *L-DCUO* retains a smoother and flatter curve. This confirms that our proposed strategy scales exceptionally well for more demanding retrieval scenarios, making it a robust solution for diverse real-world applications.

Beyond the Euclidean distance. Fig. 7 shows the search performance when the default Euclidean distance is replaced with the Inner Product (IP) metric, a measure widely adopted in Maximum Inner Product Search (MIPS) scenarios such as recommendation systems. Our method consistently outperforms all baselines across the four datasets, demonstrating that our optimization strategy is metric-agnostic and highly effective in non-Euclidean spaces. Specifically, on the OpenAI and GIST1M datasets, our approach maintains a commanding lead in QPS at 90% recall. Our method is 1.8–3.1 × faster than the SOTA graph index. Furthermore, compared to existing distance comparison operations for high-dimensional vectors, our learning-based operations achieve a 1.4–3.1 × speedup



(a) GIST1M (b) Deep100M

Figure 8: Effect of parameter L

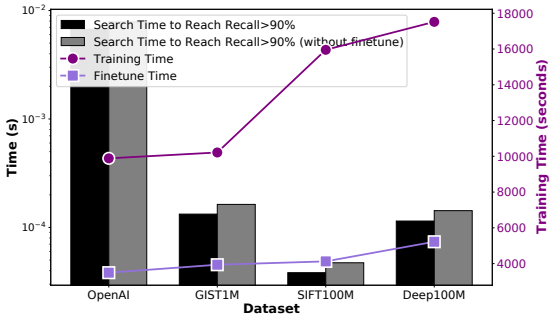


Figure 9: Effect of fine-tuning.

at the same recall level. This superior performance confirms that our learning-based distance computation operator is not only robust for standard similarity search but also offers a highly efficient solution for MIPS-based applications, significantly accelerating retrieval in recommendation contexts.

6.3 Ablation Study

Effect of parameter L. Fig. 8 shows the performance of our learning model on the GIST1M and Deep100M datasets under different values of L. A larger L implies that *L-DCUO* incorporates more sub-modules, enabling the model to capture finer-grained similarity information between vectors. Consequently, as L increases, our method achieves faster and more accurate search performance. Specifically, to reach a recall of 90%, increasing L from 2 to 6 reduces search latency on GIST1M by 0.036 ms. However, this improvement comes at the cost of increasing model training time by 4,822 seconds. Similarly, for Deep100M, the search time decreases by 0.05 ms, while the training overhead is significantly higher than that of GIST1M, increasing by 8,433 seconds.

Effect of fine-tuning. Fig. 9 presents the ablation study of the active fine-tuning strategy. It is evident that incorporating this strategy enables our method to retrieve results with greater speed and accuracy. Across the four datasets, in the absence of this strategy, the query time required to reach 90% recall increases by 17.05%, 22.89%, 23.42%, and 24.66%, respectively. In terms of computational cost, this fine-tuning stage requires between 3,492 and 5,221 seconds across all datasets, accounting for less than 30% of the total training time.

7 RELATED WORKS

We review the related works from two perspectives: *ANN Search on high-dimensional vectors* and *vector distance operations*.

ANN Search over High-Dimensional Vectors. Existing ANN search algorithms generally fall into four categories: tree-based [10, 36, 39], quantization-based [19, 22, 25], hashing-based [12, 30, 50, 58], and graph-based methods [24, 28, 44, 53, 54]. Among these, graph-based indexes have emerged as the dominant choice in modern AI-native search engines [3, 15, 45], owing to their superior search performance as demonstrated in comprehensive surveys and benchmark studies [5, 6, 38, 46]. These methods construct a proximity graph where nodes represent vectors and edges connect neighbors. Algorithms such as NSG [17] and its variants [14, 16, 56] were introduced to optimize construction and search complexity. Prominent examples include NSW [34], which utilizes consecutive insertion, and the widely adopted HNSW [35], which introduces a hierarchical structure to mitigate hubness and constrain node degrees. The success of HNSW has further inspired derivative methods like HCNNG [37], HVS [33], and SHG [23], solidifying the role of graph-based structures as the state-of-the-art solution for high-dimensional vector search.

Vector Distance Operation. Processing AkNN queries within index-based solutions relies on a massive number of vector distance computations [13, 18, 49]. Among the various operations involved, the Distance Comparison Operation (DCO) [13, 18, 52] has attracted significant attention. DCO aims to sample a subset of dimensions from the original vector to calculate and determine whether the distance between a query and a candidate vector falls within a specific threshold. While ADSampling [18] first introduced hypothesis testing to support DCOs, DADE [13] and DDC [52] later built upon this technique to propose more effective solutions. Furthermore, recent studies have integrated Machine Learning (ML) to accelerate this process. For instance, DDC [52] models the DCO as a binary classification problem. Additionally, other learning-based approaches partition the dataset and leverage distance calculations to predict the specific partition containing the final results [31, 51], thereby pruning the search space.

8 CONCLUSION

We propose a novel Learning-based Distances CompUtation Operator (*L-DCUO*) to address the computational bottleneck of distance calculations in high-dimensional vector similarity search. Our method directly computes approximate distances through learned low-dimensional embeddings, enabling seamless integration as a plug-in module into graph-based AkNN search frameworks. We design a similarity-decoupled architecture that decomposes the distance preservation task across multiple modules, each targeting a specific similarity scale. A progressive training strategy accelerates model convergence by 2.7 ×. Furthermore, an active fine-tuning strategy identifies and refines underperforming modules through contrastive learning, yielding up to 24.66% performance improvement. Extensive experiments demonstrate that when integrated into state-of-the-art graph-based indices, *L-DCUO* achieves 3.7 × speedup in search time and 3.1 × speedup in distance computations compared to existing distance comparison operators.

REFERENCES

[1] [n.d.]. Faiss (Facebook AI Similarity Search): A library for efficient similarity search and clustering of dense vectors. https://ai.meta.com/tools/faiss/.

[2] [n.d.]. Milvus: The High-Performance Vector Database Built for Scale. https://milvus.io/.

[3] [n.d.]. Qdrant: A vector similarity search engine and vector database. https://github.com/weaviate/weaviate.

[4] [n.d.]. Zilliz: Vector Database built for enterprise-grade AI applications. https://zilliz.com/.

[5] Martin Aumüller, Erik Bernhardsson, and Alexander John Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Inf. Syst.* 87 (2020), 101374.

[6] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2025. Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art. *Proc. ACM Manag. Data* 3, 1 (2025), 43:1–43:31.

[7] Yoshua Bengio. 2009. Learning Deep Architectures for AI. *Found. Trends Mach. Learn.* 2, 1 (2009), 1–127.

[8] Yoshua Bengio, Aaron C. Courville, and Pascal Vincent. 2013. Representation Learning: A Review and New Perspectives. *IEEE Trans. Pattern Anal. Mach. Intell.* 35, 8 (2013), 1798–1828.

[9] Yoshua Bengio, Yann LeCun, and Geoffrey E. Hinton. 2021. Deep learning for AI. *Commun. ACM* 64, 7 (2021), 58–65.

[10] Alina Beygelzimer, Sham M. Kakade, and John Langford. 2006. Cover trees for nearest neighbor. In *Machine Learning, Proceedings of the Twenty-Third International Conference (ICML 2006), Pittsburgh, Pennsylvania, USA, June 25-29, 2006 (ACM International Conference Proceeding Series)*, William W. Cohen and Andrew W. Moore (Eds.), Vol. 148. ACM, 97–104.

[11] Sebastian Bruch. 2024. *Foundations of Vector Retrieval.* Springer.

[12] Liwei Deng, Penghao Chen, Ximu Zeng, Yuchen Fang, Jin Chen, and Yan Zhao. 2025. Towards Accurate Distance Estimation for Distribution-Aware c-ANN Search. In *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*. IEEE, 2380–2393.

[13] Liwei Deng, Penghao Chen, Ximu Zeng, Tianfu Wang, Yan Zhao, and Kai Zheng. 2024. Efficient Data-aware Distance Comparison Operations for High-Dimensional Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 3 (2024), 812–821.

[14] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *WWW*. 577–586.

[15] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. *CoRR* abs/2401.08281 (2024).

[16] Cong Fu and Deng Cai. 2016. EFANNA : An Extremely Fast Approximate Nearest Neighbor Search Algorithm Based on kNN Graph. *CoRR* abs/1609.07228 (2016).

[17] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.

[18] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proc. ACM Manag. Data* 1, 2 (2023), 137:1–137:27.

[19] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *2013 IEEE Conference on Computer Vision and Pattern Recognition, Portland, OR, USA, June 23-28, 2013*. IEEE Computer Society, 2946–2953.

[20] Andrew V. Goldberg and Chris Harrelson. 2005. Computing the shortest path: *A* search meets graph theory. In *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2005, Vancouver, British Columbia, Canada, January 23-25, 2005*. SIAM, 156–165.

[21] Andrew V. Goldberg and Renato Fonseca F. Werneck. 2005. Computing Point-to-Point Shortest Paths from External Memory. In *Proceedings of the Seventh Workshop on Algorithm Engineering and Experiments and the Second Workshop on Analytic Algorithmics and Combinatorics, ALENEX /ANALCO 2005, Vancouver, BC, Canada, 22 January 2005*, Camil Demetrescu, Robert Sedgewick, and Roberto Tamassia (Eds.). SIAM, 26–40.

[22] Yunchao Gong, Svetlana Lazebnik, Albert Gordo, and Florent Perronnin. 2013. Iterative Quantization: A Procrustean Approach to Learning Binary Codes for Large-Scale Image Retrieval. *IEEE Trans. Pattern Anal. Mach. Intell.* 35, 12 (2013), 2916–2929.

[23] Zengyang Gong, Yuxiang Zeng, and Lei Chen. 2025. Accelerating Approximate Nearest Neighbor Search in Hierarchical Graphs: Efficient Level Navigation with Shortcuts. *Proc. VLDB Endow.* 18, 10 (2025), 3518–3530.

[24] Lars Gottesbüren, Laxman Dhulipala, Rajesh Jayaram, and Jakub Lacki. 2025. Unleashing Graph Partitioning for Large-Scale Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 6 (2025), 1649–1662.

[25] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research)*, Vol. 119. PMLR, 3887–3896.

[26] Yikun Han, Chunjiang Liu, and Pengfei Wang. 2023. A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge. *CoRR* abs/2310.11703 (2023).

[27] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *STOC*. 604–613.

[28] Wenqi Jiang, Hang Hu, Torsten Hoefler, and Gustavo Alonso. 2025. Fast Graph Vector Search via Hardware Acceleration and Delayed-Synchronization Traversal. *Proc. VLDB Endow.* 18, 11 (2025), 3797–3811.

[29] Samir Khuller, Balaji Raghavachari, and Azriel Rosenfeld. 1996. Landmarks in Graphs. *Discret. Appl. Math.* 70, 3 (1996), 217–229.

[30] Yifan Lei, Qiang Huang, Mohan S. Kankanhalli, and Anthony K. H. Tung. 2020. Locality-Sensitive Hashing Scheme based on Longest Circular Co-Substring. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo (Eds.). ACM, 2589–2599.

[31] Wuchao Li, Chao Feng, Defu Lian, Yuxin Xie, Haifeng Liu, Yong Ge, and Enhong Chen. 2023. Learning Balanced Tree Indexes for Large-Scale Vector Retrieval. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, Ambuj K. Singh, Yizhou Sun, Leman Akoglu, Dimitrios Gunopulos, Xifeng Yan, Ravi Kumar, Fatma Ozcan, and Jieping Ye (Eds.). ACM, 1353–1362.

[32] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. 2024. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. *CoRR* abs/2409.10516 (2024).

[33] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2021. HVS: Hierarchical Graph Structure Based on Voronoi Diagrams for Solving Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 15, 2 (2021), 246–258.

[34] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Inf. Syst.* 45 (2014), 61–68.

[35] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.

[36] Marius Muja and David G. Lowe. 2014. Scalable Nearest Neighbor Algorithms for High Dimensional Data. *IEEE Trans. Pattern Anal. Mach. Intell.* 36, 11 (2014), 2227–2240.

[37] Javier Alvaro Vargas Muñoz, Marcos André Gonçalves, Zanoni Dias, and Ricardo da Silva Torres. 2019. Hierarchical Clustering-Based Graphs for Large Scale Approximate Nearest Neighbor Search. *Pattern Recognit.* 96 (2019), 106970.

[38] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *VLDB J.* 33, 5 (2024), 1591–1615.

[39] Parikshit Ram and Kaushik Sinha. 2019. Revisiting kd-tree for Nearest Neighbor Search. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, Ankur Teredesai, Vipin Kumar, Ying Li, Rómer Rosales, Evimaria Terzi, and George Karypis (Eds.). ACM, 1378–1388.

[40] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2021. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *NeurIPS*, Vol. 176. 177–189.

[41] Bo Tang and Haibo He. 2015. ENN: Extended Nearest Neighbor Method for Pattern Recognition [Research Frontier]. *IEEE Comput. Intell. Mag.* 10, 3 (2015), 52–60.

[42] Yuan Tian, David Lo, and Chengnian Sun. 2012. Information Retrieval Based Nearest Neighbor Classification for Fine-Grained Bug Severity Prediction. In *WCRE*. 215–224.

[43] Michel Verleysen and Damien François. 2005. The Curse of Dimensionality in Data Mining and Time Series Prediction. In *IWANN*. 758–770.

[44] Sairaj Voruganti and M. Tamer Özsu. 2025. MIRAGE-ANNS: Mixed Approach Graph-based Indexing for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3 (2025), 188:1–188:27.

[45] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, Guoliang Li, Zhanhuai Li, Stratos Idreos, and Divesh Srivastava (Eds.). ACM, 2614–2627.

[46] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 14, 11 (2021), 1964–1978.

[47] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *PVLDB* 14, 11 (2021), 1964–1978.

[48] Qitong Wang and Themis Palpanas. 2021. Deep Learning Embeddings for Data Series Similarity Search. In *KDD '21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, Singapore, August 14-18, 2021*, Feida Zhu, Beng Chin Ooi, and Chunyan Miao (Eds.). ACM, 1708–1716.

[49] Zeyu Wang, Haoran Xiong, Qitong Wang, Zhenying He, Peng Wang, Themis Palpanas, and Wei Wang. 2024. Dimensionality-Reduction Techniques for Approximate Nearest Neighbor Search: A Survey and Evaluation. *IEEE Data Eng. Bull.* 48, 3 (2024), 63–80.

[50] Jiuqi Wei, Botao Peng, Xiaodong Lee, and Themis Palpanas. 2024. DET-LSH: A Locality-Sensitive Hashing Scheme with Dynamic Encoding Tree for Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 17, 9 (2024), 2241–2254.

[51] Wenqing Wei, Defu Lian, Qingshuai Feng, and Yongji Wu. 2025. Robust Tree-based Learned Vector Index with Query-aware Repartitioning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.2, KDD 2025, Toronto ON, Canada, August 3-7, 2025*, Luiza Antonie, Jian Pei, Xiaohui Yu, Flavio Chierichetti, Hady W. Lauw, Yizhou Sun, and Srinivasan Parthasarathy (Eds.). ACM, 3134–3143.

[52] Mingyu Yang, Wentao Li, Jiabao Jin, Xiaoyao Zhong, Xiangyu Wang, Zhitao Shen, Wei Jia, and Wei Wang. 2025. Effective and General Distance Computation for Approximate Nearest Neighbor Search. In *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*. IEEE, 1098–1110.

[53] Shuo Yang, Jiadong Xie, Yingfan Liu, Jeffrey Xu Yu, Xiyue Gao, Qianru Wang, Yanguo Peng, and Jiangtao Cui. 2025. Revisiting the Index Construction of Proximity Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 6 (2025), 1825–1838.

[54] Ziqi Yin, Jianyang Gao, Pasquale Balsebre, Gao Cong, and Cheng Long. 2025. DEG: Efficient Hybrid Vector Search Using the Dynamic Edge Navigation Graph. *Proc. ACM Manag. Data* 3, 1 (2025), 29:1–29:28.

[55] Hailin Zhang, Penghao Zhao, Xupeng Miao, Yingxia Shao, Zirui Liu, Tong Yang, and Bin Cui. 2023. Experimental Analysis of Large-scale Learnable Vector Storage Compression. *PVLDB* 17, 4 (2023), 808–822.

[56] Wan-Lei Zhao, Hui Wang, Peng-Cheng Lin, and Chong-Wah Ngo. 2022. On the Merge of k-NN Graph. *IEEE Trans. Big Data* 8, 6 (2022), 1496–1510.

[57] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proc. VLDB Endow.* 16, 8 (2023), 1979–1991.

[58] Bolong Zheng, Xi Zhao, Lianggui Weng, Nguyen Quoc Viet Hung, Hang Liu, and Christian S. Jensen. 2020. PM-LSH: A Fast and Accurate LSH Framework for High-Dimensional Approximate NN Search. *Proc. VLDB Endow.* 13, 5 (2020), 643–655.